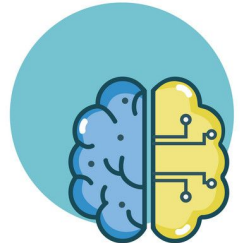


INTRODUCTION TO MACHINE LEARNING

DECISION TREES



Elisa Ricci



TODAY

Decision Trees

- Simple, intuitive and powerful supervised learning algorithm
- Explainable, similar to how humans make decisions
- Very popular! One of the most widely used learning algorithms in Kaggle competitions

Introduce **ensembles**, a key idea in ML

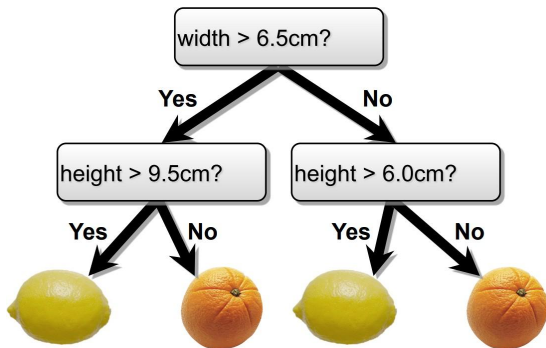
Information Theoretic concepts (e.g., entropy) and Gini index

Random Forests

DECISION TREES

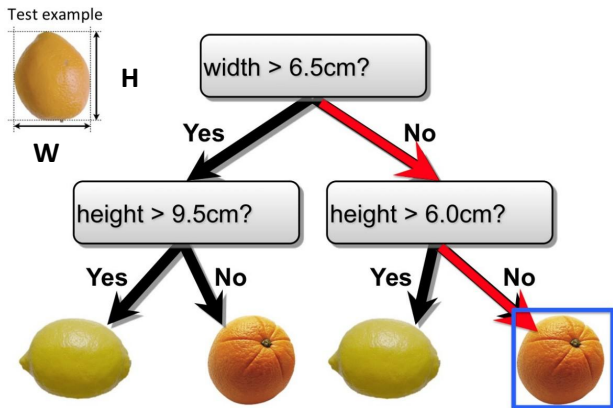
Decision trees make predictions by recursively splitting on different attributes according to a **tree structure**

Example: classifying fruit as an orange or lemon based on the attributes (or features) height and width



DECISION TREES

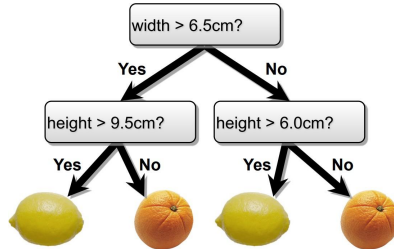
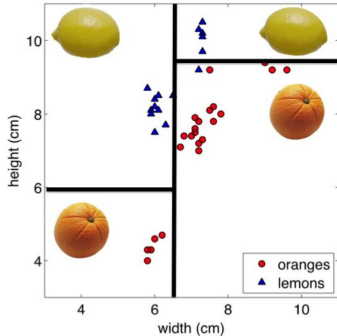
Lets say the test sample has the following features: $W = 5.3$ cm, $H = 5$ cm



DECISION TREES

For **continuous attributes**, splitting is based on less than or greater than some threshold

Thus, the input space is divided into regions with **boundaries** parallel to the axes

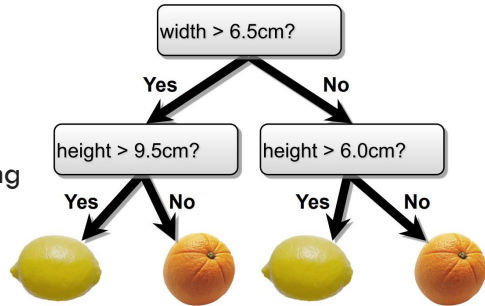


DECISION TREES

What is the “learning” part in decision trees?

Given a training dataset, we need to learn:

- Which **attribute** to choose for splitting (e.g., width)
- What **value of that attribute** to use for splitting (e.g., width > 6.5 cm)



DECISION TREES: EXAMPLE WITH DISCRETE INPUTS

What if the attributes are **discrete**?

The tables below describe the following:

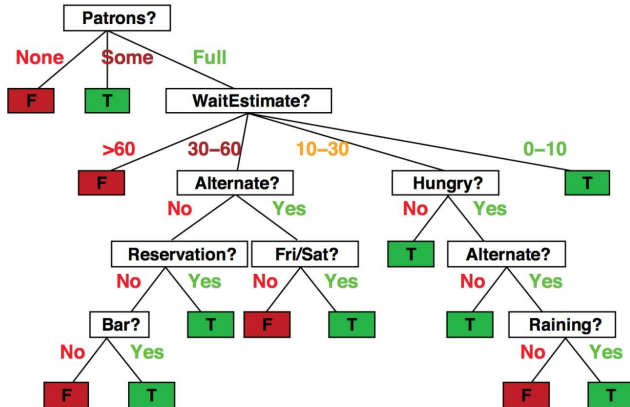
- Left: Data points
- Right: Attributes

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	$y_{12} = \text{Yes}$

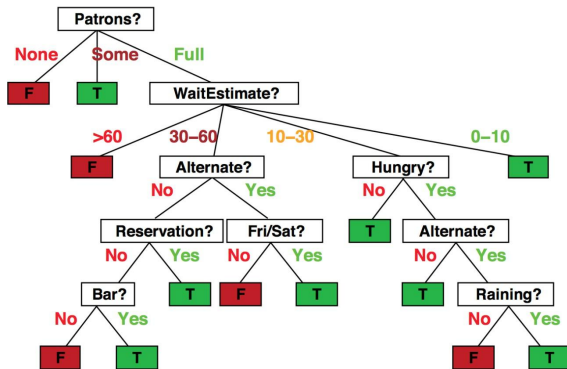
1.	Alternate: whether there is a suitable alternative restaurant nearby.
2.	Bar: whether the restaurant has a comfortable bar area to wait in.
3.	Fri/Sat: true on Fridays and Saturdays.
4.	Hungry: whether we are hungry.
5.	Patrons: how many people are in the restaurant (values are None, Some, and Full).
6.	Price: the restaurant's price range (\$, \$\$, \$\$\$).
7.	Raining: whether it is raining outside.
8.	Reservation: whether we made a reservation.
9.	Type: the kind of restaurant (French, Italian, Thai or Burger).
10.	WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

DECISION TREES: EXAMPLE WITH DISCRETE INPUTS

Possible tree to decide whether to wait (T) or not (F)



DECISION TREES: EXAMPLE WITH DISCRETE INPUTS



Internal nodes test attributes

Branching is determined by attribute value

Leaf nodes are outputs (or predictions)

DECISION TREE: CLASSIFICATION AND REGRESSION

Each path from root to a leaf defines a region R_m of input space

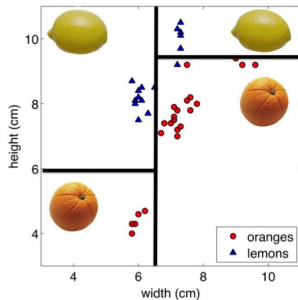
Training examples that fall into the region R_m

$$\{(\mathbf{x}^{(m_1)}, t^{(m_1)}), \dots, (\mathbf{x}^{(m_k)}, t^{(m_k)})\}$$

Classification tree:

- discrete output, i.e., $y \in \{1, \dots, C\}$
- leaf value: y^m typically set to the most common value (or **majority vote**) in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$ i.e.,

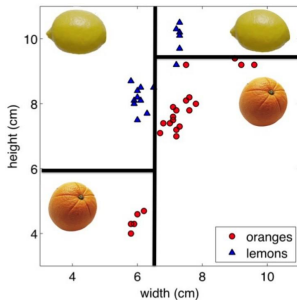
$$y^m \leftarrow \operatorname{argmax}_{t \in \{1, \dots, C\}} \sum_{m_i} \mathbb{1}\{t = t^{(m_i)}\}$$



DECISION TREE: CLASSIFICATION AND REGRESSION

Regression tree:

- continuous output, i.e. $y \in \mathbb{R}$
- leaf value y^m typically set to the mean value in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$



Note: We will only focus on classification

HOW DO WE LEARN A DECISION TREE?

How do we construct a useful decision tree?

We want to find a “simple” tree that explains data well.

Desiderata are:

- Simple or small: Minimal number of nodes
- There should be enough samples per region

LEARNING DECISION TREES

Learning the simplest (smallest) decision tree that correctly classifies training set is a NP complete problem
(No efficient solutions can be found)

LEARNING DECISION TREES

Resort to a greedy heuristic

Start with an empty decision tree and complete training set

- Split (i.e., partition dataset) on the “best” attribute
- Recursive on subpartitions

When should we stop?

Which attribute is the “best”?

- We define a notion of gain of a split
- Gain is defined based on change in some criteria before and after a split
- Various notions of gains

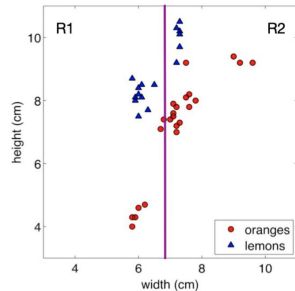
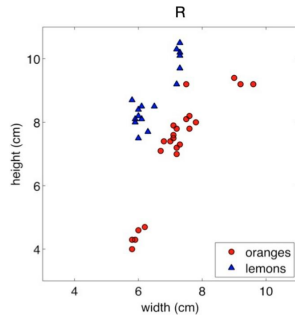
LEARNING DECISION TREES

Which attribute is the “best”?

Let us choose the accuracy (i.e., misclassification error L – the number of incorrect classifications) as the criterion, and define the accuracy gain

Let us define **accuracy gain**:

- Suppose we have a region R . Denote the misclassification error of that region as $L(R)$
- We split R into two regions R_1 and R_2 based on some attribute (e.g., width)
- What is the error of the two regions after the split?



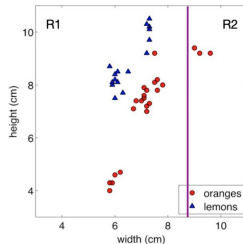
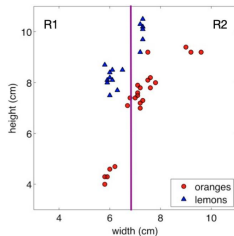
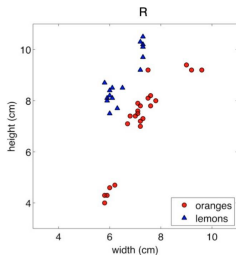
LEARNING DECISION TREES

Misclassification error **before** the split: $L(R)$

Misclassification error **after** the split:

$$\frac{|R_1|}{|R|}L(R_1) + \frac{|R_2|}{|R|}L(R_2),$$

Where: $\frac{|R_i|}{|R|}$ is the ratio of samples on the i-th split

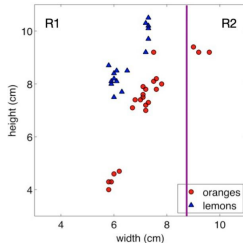
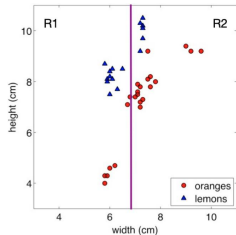
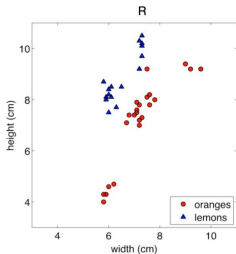


LEARNING DECISION TREES

Accuracy gain is:

$$L(R) - \frac{|R_1|L(R_1) + |R_2|L(R_2)}{|R|}$$

Note that different splits lead to different accuracy

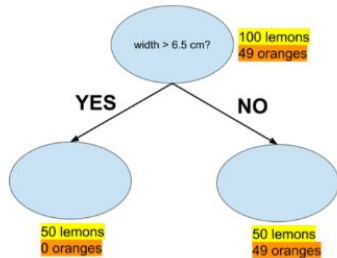


CHOOSING A GOOD SPLIT

Accuracy is not always a good measure for deciding the split. Why?

Is this split good? Accuracy gain is:

$$L(R) - \frac{|R_1|L(R_1) + |R_2|L(R_2)}{|R|} = \frac{49}{149} - \frac{50 \times 0 + 99 \times \frac{49}{99}}{149} = 0$$



Although we have reduced our uncertainty about whether a fruit is a lemon, the accuracy gain is zero!

CHOOSING A GOOD SPLIT

We can use **uncertainty** as the criterion, and use **gain in the certainty** (or gain in the reduction of uncertainty) to decide the split

How can we quantify uncertainty in prediction for a given leaf node?

- All examples in leaf have the same class: **good** (low uncertainty)
- Each class has the same number of examples in leaf: **bad** (high uncertainty)

Idea: Take help of **information theory** to measure uncertainty

Information

Sum

Probability of symbol

$H = -\sum_x p(x) \log p(x)$

Symbols

Base-2 logarithm

The diagram shows the formula $H = -\sum_x p(x) \log p(x)$ with several labels and leader lines. 'Information' points to 'H'. 'Sum' points to the summation symbol $\sum$. 'Probability of symbol' has two lines pointing to the two 'p(x)' terms. 'Symbols' has two lines pointing to the 'x' and the first 'p(x)'. 'Base-2 logarithm' has two lines pointing to the 'log' and the second 'p(x)'.

BASICS OF INFORMATION THEORY

FLIPPING TWO DIFFERENT COINS

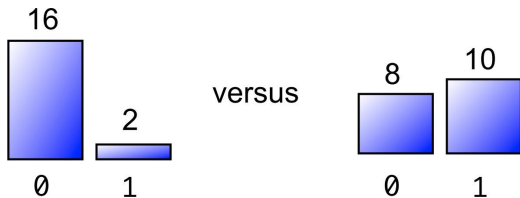
Q: Which coin is more uncertain?

Sequence 1:

0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Sequence 2:

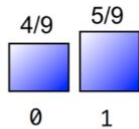
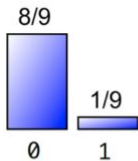
0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?



QUANTIFYING UNCERTAINTY

Entropy is a measure of expected “surprise”. How uncertain are we of the value of a draw from this distribution?

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in X} p(x) \log_2 p(x)$$



$$-\left(\frac{8}{9} \log_2 \frac{8}{9} + \frac{1}{9} \log_2 \frac{1}{9}\right) \approx \frac{1}{2}$$

$$-\left(\frac{4}{9} \log_2 \frac{4}{9} + \frac{5}{9} \log_2 \frac{5}{9}\right) \approx 0.99$$

QUANTIFYING UNCERTAINTY

Entropy is a measure of expected “surprise”. How uncertain are we of the value of a draw from this distribution?

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in X} p(x) \log_2 p(x)$$

Unit = bits

A fair coin flip has 1 bit of entropy

MORE ON ENTROPY

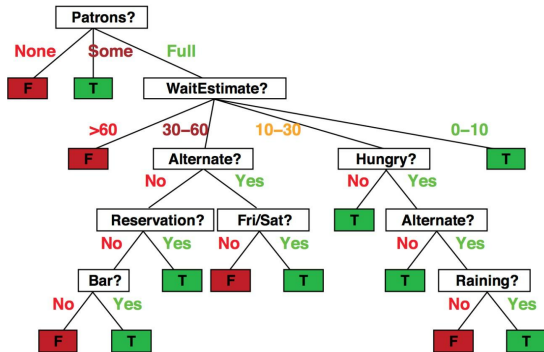
“High Entropy”:

- Variable has a uniform-like distribution
- Flat histogram
- Values sampled from it are less predictable

“Low Entropy”:

- Distribution of variable has peaks and valleys
- Histogram has lows and highs
- Values sampled from it are more predictable

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in X} p(x) \log_2 p(x)$$



BACK TO DECISION TREES

INFORMATION GAIN

Information Gain measures the **expected reduction in entropy**

Let us define information gain:

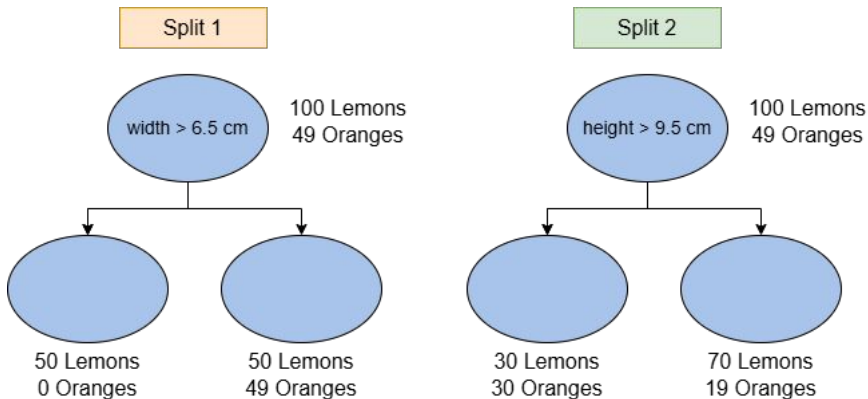
- Entropy of region R or parent node before the split: $H(R)$
- Entropy of regions R_1 and R_2 , or leaf nodes after the split: $H(R_1)$ and $H(R_2)$
- Information gain is defined as:

$$IG = H(R) - \left\{ \frac{|R_1|}{|R|} H(R_1) + \frac{|R_2|}{|R|} H(R_2) \right\}$$

The feature and the corresponding value with the **highest information gain** will be selected for splitting

CHOOSING A GOOD SPLIT USING INFORMATION GAIN

Q: Using information gain find which is a better split?



CHOOSING A GOOD SPLIT USING INFORMATION GAIN

Entropy of root node

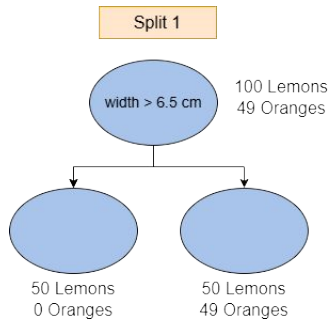
$$H(\text{root}) = -\left(\frac{100}{149} \log_2 \frac{100}{149} + \frac{49}{149} \log_2 \frac{49}{149}\right) \approx 0.910$$

Entropy of left child

$$H(\text{left}) = -\left(\frac{50}{50} \log_2 \frac{50}{50} + \frac{0}{50} \log_2 \frac{0}{50}\right) = 0$$

Entropy of right child

$$H(\text{right}) = -\left(\frac{50}{99} \log_2 \frac{50}{99} + \frac{49}{99} \log_2 \frac{49}{99}\right) \approx 1$$



CHOOSING A GOOD SPLIT USING INFORMATION GAIN

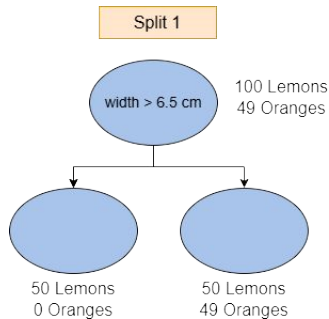
Information gain

$$IG = H(\text{root}) - \left\{ \frac{|R_1|}{|R|} H(\text{left}) + \frac{|R_2|}{|R|} H(\text{right}) \right\},$$

$$\frac{|R_1|}{|R|} = \frac{50}{149} \quad \frac{|R_2|}{|R|} = \frac{99}{149}$$

Substituting the values we get:

$$\begin{aligned} IG &= 0.910 - \left(\frac{50}{149} \times 0 + \frac{99}{149} \times 1 \right) \\ &= 0.24 \end{aligned}$$



Choosing split 1 will lead to 0.24 units of reduction of uncertainty

CHOOSING A GOOD SPLIT USING INFORMATION GAIN

Entropy of **root node** remains unchanged:

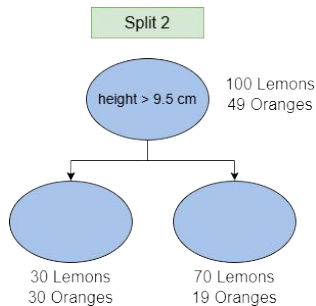
$$H(\text{root}) \approx 0.910$$

Entropy of **left child**

$$H(\text{left}) = -\left(\frac{30}{60} \log_2 \frac{30}{60} + \frac{30}{60} \log_2 \frac{30}{60}\right) = 1$$

Entropy of **right child**

$$H(\text{right}) = -\left(\frac{70}{89} \log_2 \frac{70}{89} + \frac{19}{89} \log_2 \frac{19}{89}\right) \approx 0.748$$



CHOOSING A GOOD SPLIT USING INFORMATION GAIN

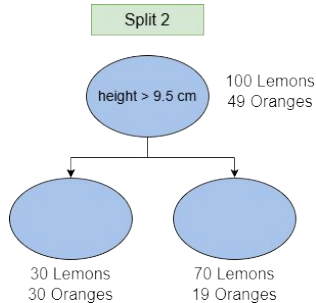
Information gain

$$IG = H(\text{root}) - \left\{ \frac{|R_1|}{|R|} H(\text{left}) + \frac{|R_2|}{|R|} H(\text{right}) \right\},$$

$$\frac{|R_1|}{|R|} = \frac{60}{149}, \quad \frac{|R_2|}{|R|} = \frac{89}{149}$$

Substituting the values we get:

$$\begin{aligned} IG &= 0.910 - \left(\frac{60}{149} \times 0 + \frac{89}{149} \times 1 \right) \\ &= 0.001 \end{aligned}$$



Choosing split 2 will lead to 0.001 units of reduction of uncertainty. Thus, split 1 is a better split

GINI INDEX

Gini Index is another widely used metric for choosing a good split in decision trees

Gini Index measures the “impurity” (or non-homogeneity) at a given node as:

$$\text{GINI}(R) = 1 - \sum_j [p(j|R)]^2,$$

where, $p(j|R)$ is the class frequency of class j in region (or node) R .

- Intuitively, it aims to measure the probability of misclassifying a randomly chosen element
- Greater the value of the Gini Index, the greater the chances of having misclassifications
- Thus, we will favour splits with lower Gini value

CHOOSING A GOOD SPLIT USING GINI INDEX

We will directly compute the GINI Index at the children nodes

GINI of **left child**:

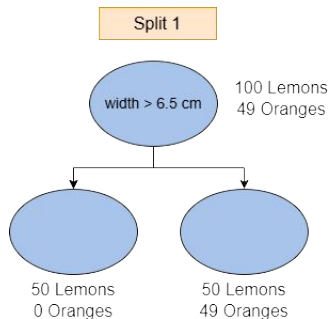
$$\text{GINI}(\text{left}) = 1 - \left[\left(\frac{50}{50} \right)^2 + \left(\frac{0}{50} \right)^2 \right] = 1 - 1 = 0$$

GINI of **right child**:

$$\text{GINI}(\text{right}) = 1 - \left[\left(\frac{50}{99} \right)^2 + \left(\frac{49}{99} \right)^2 \right] \approx 0.5$$

GINI of **Split 1**:

$$\text{GINI}(\text{split}) = \frac{50}{149} \times 0 + \frac{99}{149} \times 0.5 = 0.332$$



CHOOSING A GOOD SPLIT USING GINI INDEX

GINI of **left child**:

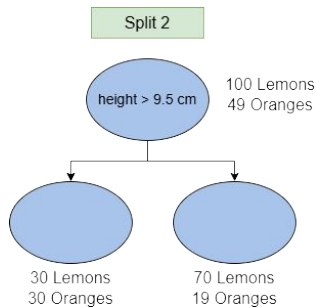
$$\text{GINI}(\text{left}) = 1 - \left[\left(\frac{30}{60}\right)^2 + \left(\frac{30}{60}\right)^2\right] = 0.5$$

GINI of **right child**:

$$\text{GINI}(\text{left}) = 1 - \left[\left(\frac{70}{89}\right)^2 + \left(\frac{19}{89}\right)^2\right] \approx 0.335$$

GINI of **Split2**:

$$\text{GINI}(\text{split}) = \frac{60}{149} \times 0.5 + \frac{89}{149} \times 0.335 = 0.401$$



We favour Split 1 because it has lower GINI Index (or lower impurity)

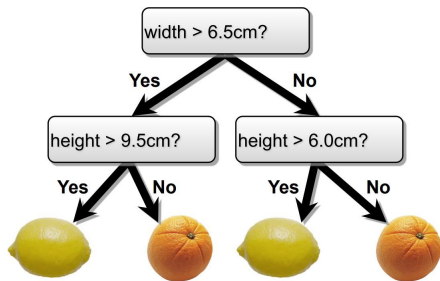
CONSTRUCTING DECISION TREES

At each level, one must choose:

- which **attribute** (or feature) to split
- what **value** of the attribute to split

Choose attribute & value that gives:

- **highest accuracy** gain OR
- **highest information** gain OR
- **lowest** GINI index

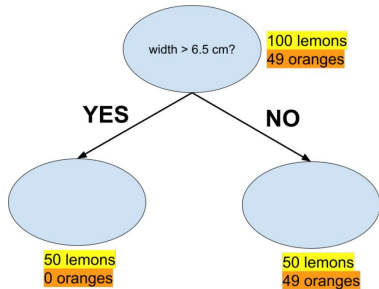


DECISION TREE CONSTRUCTION ALGORITHM

Start with empty decision tree and complete training set

- Split on the most informative attribute
- Recurse on subpartitions

Possible **termination condition**: end if all examples in current subpartition share the same class



WHAT MAKES A GOOD TREE?

Not too small: need to handle important but possibly subtle distinctions in data

Not too big:

- Avoid overfitting training examples
- We need enough samples in each region to confidently determine the output
- Computational efficiency (avoid redundant, spurious attributes)
- Human interpretability

“Occam’s Razor” says *“With all else being equal, a simpler model is preferred over a more complex model”*

- Find the simplest hypothesis that fits the observations

We desire small trees with informative nodes near the root

DECISION TREE MISCELLANY

Problems:

- You have exponentially less data at lower levels
- A large tree can overfit the data
- Decision Trees do not necessarily reach the global minima
- Mistakes at top-level propagate down tree

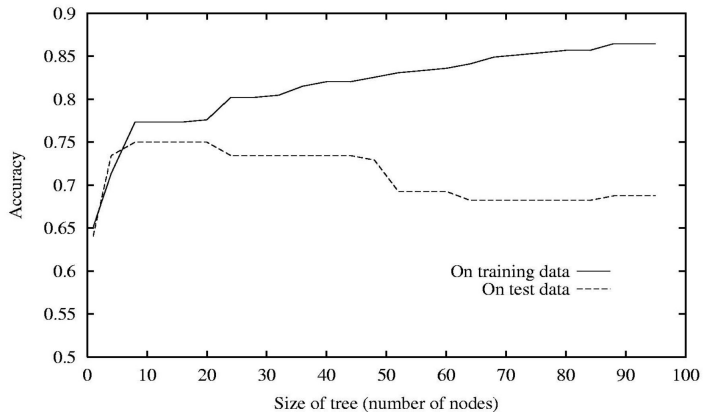
Decision trees can also be used for regression on real-valued outputs

- Choose splits to minimize squared error, rather than maximize information gain

DECISION TREES AND OVERFITTING

- Decision trees have a structure that is determined by the data
- As a result they are flexible and can easily fit the training set, with high risk of **overfitting**
- Standard techniques to improve generalization apply also to decision trees (e.g. data augmentation, ensembling)
- A technique to reduce complexity a posteriori is called **pruning**
- Pruning of the decision tree is done by replacing a whole subtree by a leaf node.

DECISION TREES AND OVERFITTING



COMPARISON TO KNN

Advantages of Decision Trees over KNN

- Good with discrete attributes
- Easily deals with missing values (just treat as another value)
- Robust to scale of inputs
- Good when there are lots of attributes, but only a few are important
- Fast at test time
- More interpretable

SUMMARY OF DT

Widely used in practice

Strengths

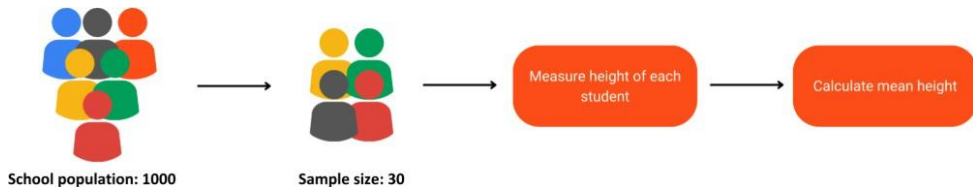
- Fast and simple to implement
- Can convert to rules

Weaknesses

- Univariate feature split
- Requires fixed-length feature vectors
- Non-incremental (i.e., batch method)

BOOTSTRAPPING – FOUNDATION OF RANDOM FORESTS

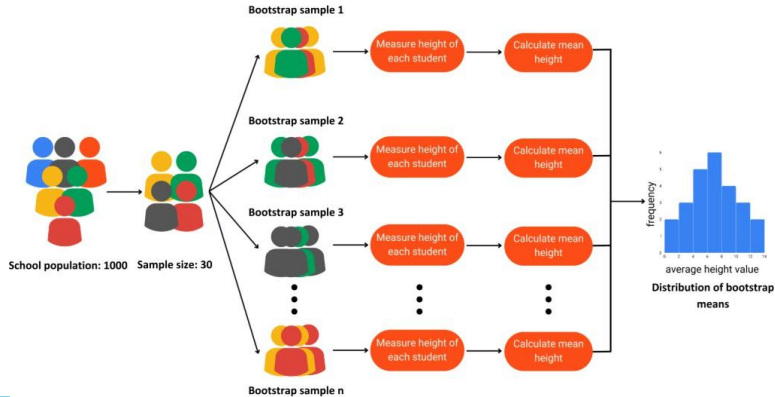
Problem: We want to find the mean height of all the students in a school



Issue: The mean height calculated from this single sample might not be a reliable estimate of the population mean due to sampling variability

Similarly, the main drawbacks of Decision Trees is that they suffer from being high-variance estimators

BOOTSTRAPPING – FOUNDATION OF RANDOM FORESTS

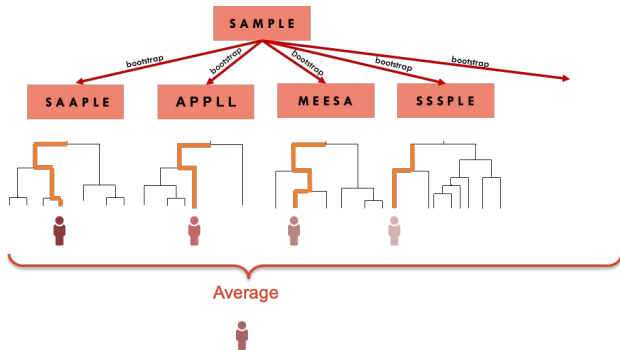


Bootstrapping is a resampling technique that involves repeatedly drawing samples from the dataset with replacement

Creates multiple datasets that introduce variability, reducing overfitting in models

BAGGING: COMBINING MULTIPLE DECISION TREES

Bagging (or Bootstrap Aggregating) involves training multiple decision trees on different bootstrapped samples and averaging their outputs (for regression) or majority voting (for classification)



Reduces variance and improves model stability

FEATURE RANDOMNESS IN RANDOM FORESTS

- In addition to bootstrapping, Random Forests introduce **feature randomness** – each tree is trained on a random subset of features
- This decreases the correlation between each DT and thus increasing its predictive accuracy, on average
- In other words, avoid very strong predictive features that lead to similar splits in trees

RANDOM FOREST – THE FULL ALGORITHM

Random Forest

1. Draw multiple bootstrapped datasets from the original data
2. Train a DT on each dataset using a random subset of ($\sqrt{d}$) features at each split
3. Aggregate predictions
 - ▶ Classification: Majority vote
 - ▶ Regression: Average predictions

Advantages:

- Reduces overfitting compared to a single decision tree
- Works well on large datasets with high-dimensional features
- Robust to missing values and noise

APPLICATIONS

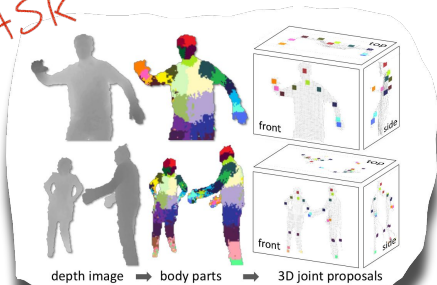
Random forests are in XBox



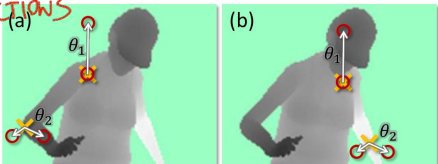
J. Shotton, A. Fitzgibbon, M. Cook, T. Sharp, M. Finocchio, R. Moore, A. Kipman, A. Blake. Real-Time Human Pose
https://www.youtube.com/watch?v=QPYf6pXe_4Q

KINECT AND RANDOM FORESTS

TASK



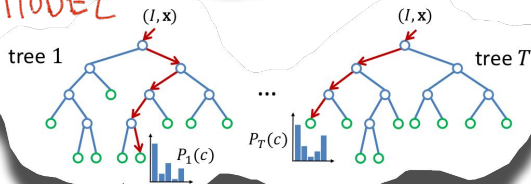
SPLIT
FUNCTIONS



DATA SET



MODEL



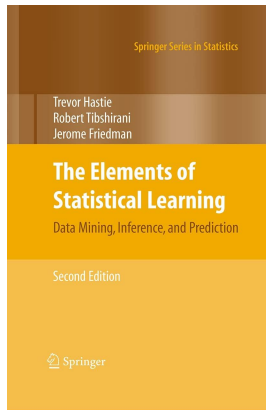
REFERENCES AND LINKS

For additional reading materials refer to the book
“The Elements of Statistical Learning” (Second
Edition)

- Chapter 9.2
- Chapter 8.7

Slide courtesy: Some of the slides have been
adapted from F. Amir

<https://hastie.su.domains/ElemStatLearn/>



QUESTIONS?

